

# **INSTITUICAO DE ENSINO**

**Curso: Programacao Orientada a Objetos**

## **TRACKFIT POO**

**Sistema de Rastreamento de Treinos e Cargas**

Grupo: preencher os nomes dos 8 integrantes

Professor(a): preencher

**2026**

# SUMARIO

- 1 Introducao
- 2 Modelagem Conceitual
- 3 Aplicacao dos Conceitos de POO
- 4 Uso de Bibliotecas
- 5 Manual do Usuario
- 6 Conclusao

# 1 INTRODUCAO

O tema escolhido para o projeto foi um sistema de rastreamento de treinos e cargas. Em academias, clubes e treinos individuais, uma dificuldade comum é registrar o que foi executado, calcular a carga aplicada e acompanhar a evolução do atleta ao longo do tempo. O TrackFit POO organiza esse processo por meio do cadastro de treinadores, atletas, exercícios e treinos.

O sistema foi planejado para atender aos critérios do projeto final de POO: diversidade de entidades, relacionamentos entre classes, uso de herança, composição, agregação, polimorfismo, tratamento de exceções, persistência de dados e interface gráfica com javax.swing.

## Atendimento aos requisitos do enunciado

| Requisito                  | Como foi atendido                                                        |
|----------------------------|--------------------------------------------------------------------------|
| 3 a 5 entidades principais | Atleta, Treinador, Exercício, Treino e ItemTreino.                       |
| Herança                    | Pessoa -> Atleta/Treinador; Exercício -> ExercícioForça/ExercícioCardio. |
| Composição e agregação     | Treino é composto por ItemTreino; Atleta agrega Treinador.               |
| Polimorfismo               | Cada tipo de Exercício calcula carga com sua própria fórmula.            |
| Persistência               | Dados salvos em data/trackfit.ser por serialização.                      |
| Interface gráfica          | Tela Swing com abas para cadastros, treinos e relatórios.                |

## 2 MODELAGEM CONCEITUAL

A modelagem separa as responsabilidades entre classes de dominio, servico, persistencia e interface. As classes de dominio representam os dados centrais do problema. O servico concentra regras de negocio, validacoes e chamadas de persistencia. A interface Swing apenas coleta dados do usuario e apresenta resultados.

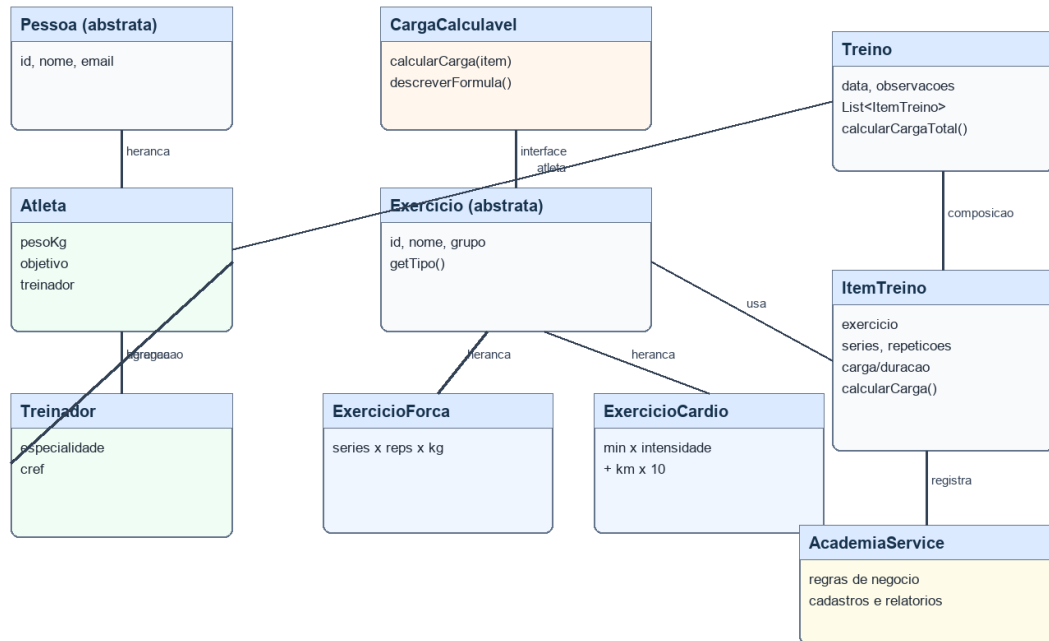


Figura 1 - Diagrama de classes UML simplificado do TrackFit POO.

### 3 APLICACAO DOS CONCEITOS DE POO

Heranca e classes abstratas aparecem na estrutura de pessoas e exercicios. A classe Pessoa concentra atributos comuns; Atleta e Treinador especializam esses dados. A classe Exercicio define a base do calculo de carga, enquanto ExercicioForca e ExercicioCardio implementam comportamentos diferentes.

```
public abstract class Exercicio implements Serializable, CargaCalculavel {
    private int id;
    private String nome;
    private String grupo;

    public abstract String getTipo();
}
```

O polimorfismo ocorre quando ItemTreino chama calcularCarga no exercicio associado. A chamada e a mesma, mas a formula executada depende do objeto concreto: forca ou cardio.

```
public class ExercicioForca extends Exercicio {
    @Override
    public double calcularCarga(ItemTreino item) {
        return item.getSeries() * item.getRepeticoes() * item.getCargaKg();
    }
}

public class ExercicioCardio extends Exercicio {
    @Override
    public double calcularCarga(ItemTreino item) {
        return (item.getDuracaoMinutos() * item.getIntensidade())
            + (item.getDistanciaKm() * 10.0);
    }
}
```

A composicao aparece em Treino, que possui uma lista de ItemTreino. Se um treino e excluido, seus itens deixam de existir no contexto do sistema. A agregacao aparece no relacionamento entre Atleta e Treinador, pois ambos podem existir independentemente.

```
public class Treino implements Serializable {
    private Atleta atleta;
    private List<ItemTreino> itens;

    public double calcularCargaTotal() {
        double total = 0.0;
        for (ItemTreino item : itens) {
            total += item.calcularCarga();
        }
        return total;
    }
}
```

As excecoes customizadas DadosInvalidosException e EntidadeNaoEncontradaException tornam as validacoes mais claras para a interface, que exibe mensagens ao usuario sem misturar regras de negocio com codigo visual.

### 4 USO DE BIBLIOTECAS

O projeto utiliza apenas recursos da Java Standard Edition, evitando dependências externas. A biblioteca javax.swing é usada na interface gráfica. As coleções da API java.util armazenam listas de atletas, treinadores, exercícios e treinos. A API java.time representa datas de treino com LocalDate. A persistência usa java.io com ObjectOutputStream e ObjectInputStream.

## 5 MANUAL DO USUÁRIO

Para executar o sistema, instale um JDK 8 ou superior, abra a pasta do projeto e rode compile.bat. Depois execute run.bat. Ao abrir, o sistema cria alguns dados de exemplo e salva novas informações no arquivo data/trackfit.ser.

| ID | Data       | Atleta       | Itens | Carga  | Nível      |
|----|------------|--------------|-------|--------|------------|
| 1  | 2026-07-14 | Carlos Souza | 2     | 1380.0 | Alta       |
| 2  | 2026-07-12 | Marina Rocha | 1     | 260.0  | Leve       |
| 3  | 2026-07-10 | Carlos Souza | 3     | 1900.0 | Muito alta |

Figura 2 - Tela principal do sistema com registro de treino e listagem de cargas.

Fluxo de uso: cadastrar treinadores, cadastrar atletas vinculando-os a um treinador, cadastrar exercícios de força ou cardio, acessar a aba Treinos, selecionar atleta e exercício, preencher os dados do item, adicionar o item ao treino e finalizar em Registrar treino. A aba Relatórios apresenta a carga acumulada por atleta e os últimos treinos cadastrados.

Link do GitHub: preencher pelo grupo após subir o projeto completo.

## 6 CONCLUSÃO

O TrackFit POO demonstra a aplicação prática dos principais conceitos de Programação Orientada a Objetos em um problema real: o acompanhamento de treinos e cargas. A separação entre domínio, serviço, persistência e interface facilita manutenção e evolução. O sistema pode ser ampliado futuramente com metas semanais, gráficos de desempenho, login de usuários e exportação de relatórios.

## **REFERENCIAS**

ORACLE. Java Platform, Standard Edition Documentation. Disponivel na documentacao oficial da linguagem Java.

Material da disciplina de Programacao Orientada a Objetos e enunciado do Projeto Final de POO.